Validation is an important part of the model-binding process in ASP.NET Core. In chapter 7 you learned that minimal APIs don't have any validation built in, so you're free to choose whichever framework you like. I demonstrated using `DataAnnotations`, but you could easily choose a different validation framework.

In Razor Pages and MVC, however, the `DataAnnotations` validation framework is built into ASP.NET Core. You can apply `DataAnnotations` attributes to properties of your binding models to define your requirements, and ASP.NET Core automatically validates them. In section 32.3 we even created a custom validation attribute.

But ASP.NET Core is flexible. You can replace whole chunks of the Razor Pages and MVC frameworks if you like. The validation system is one such area that many people choose to replace.

FluentValidation (https://fluentvalidation.net) is a popular alternative validation framework for ASP.NET Core. It is a mature library, with roots going back well before ASP.NET Core was conceived of. With FluentValidation you write your validation code separately from your binding model code. This gives several advantages:

- You're not restricted to the limitations of `Attributes`, such as the dependency injection problem we had to work around in listing 32.9.
- It's much easier to create validation rules that apply to multiple properties, such as to ensure that an `EndDate` property contains a later value than a `StartDate` property. Achieving this with `DataAnnotations` attributes is possible but difficult.